The response body depends on whether Catalog Service is running or not, but that doesn't matter in this example. The interesting aspect to notice is the HTTP headers of the response. They show the rate limiter's configuration and the number of remaining requests allowed within the time window (1 second):

```
HTTP/1.1 200 OK
Content-Type: application/json
X-RateLimit-Burst-Capacity: 20
X-RateLimit-Remaining: 19
X-RateLimit-Replenish-Rate: 10
X-RateLimit-Requested-Tokens: 1
```

You might not want to expose this information to clients in cases where the information could help bad actors craft attacks against your system. Or you might need different header names. Either way, you can use the `spring.cloud.gateway.redis-rate-limiter` property group to configure that behavior. When you're done testing the application, stop it with Ctrl-C.

NOTE When the rate limiter pattern is combined with other patterns like time limiters, circuit breakers, and retries, the rate limiter is applied first. If a user's request exceeds the rate limit, it is rejected right away.

Redis is an efficient data store ensuring fast data access, high availability, and resilience. In this section, we used it to provide storage for the rate limiters, and the next section will show you how to use it in another common scenario: session management.

9.4 Distributed session management with Redis